

Blockchain-Based Healthcare Record Verification System

What Phase 2 actually built — every running process, every Docker image,
every line of our own code, and how they talk to each other

Reference document — Phase 2 checkpoint
Generated 2026-09-22

0. How to read this document

Everything below is grounded in the actual running system on this machine right now — real container names, real ports, real file sizes, real file paths in this repository. Nothing here is a generic textbook description of "how Fabric works" — it's "what is literally running, right now, because of specific files we wrote."

Two labels are used throughout:

OUR CODE — something we wrote, in this repository, that you can open and read end to end.

HYPERLEDGER'S CODE — prebuilt software published by the Hyperledger Fabric project. We never see or edit its source; we only configure it and run it as a Docker container. Its actual send/receive logic lives in the Fabric project's own Go source code on GitHub, not in this repository.

Keeping that distinction straight is the single most useful thing for understanding what's happening: **almost everything running is Hyperledger's code, doing exactly what it always does. Our job was to configure it correctly and write one small piece of application logic (the chaincode).**

0.1 The one-sentence version

We are running 11 separate programs (as 11 Docker containers) that together simulate 4 independent organizations sharing one private blockchain. We wrote almost none of the code that runs inside 8 of those containers — that's Hyperledger's software. We wrote configuration files that tell that software who the 4 organizations are and how to find each other, plus about 45 lines of actual application logic (the chaincode) that runs in the other 3 containers.

1. The full picture: what is running right now

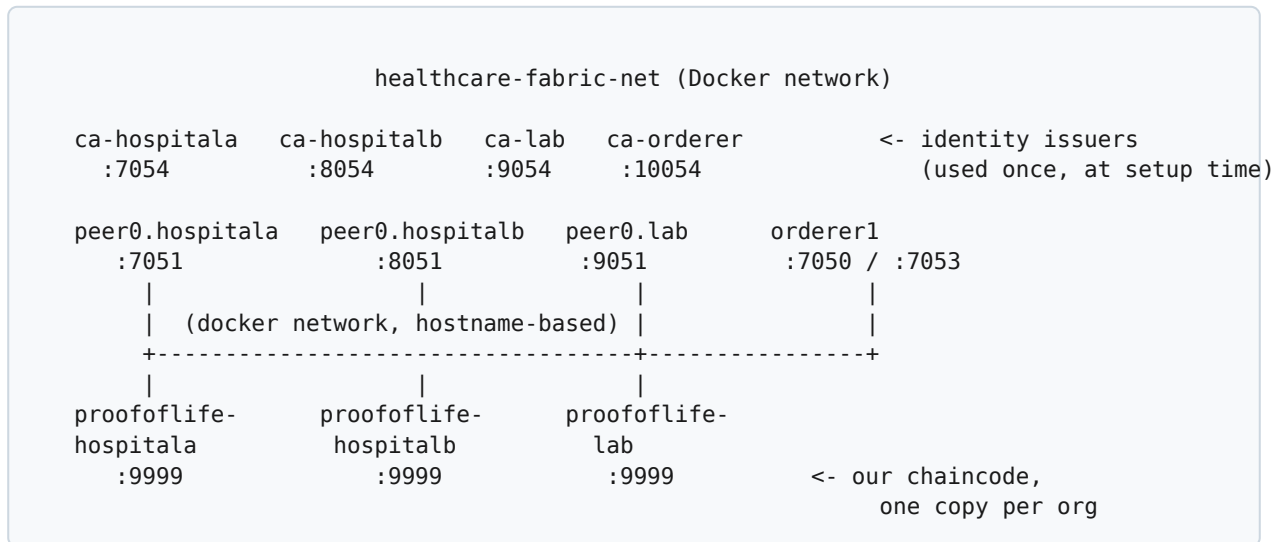
Run `docker ps` right now and, ignoring unrelated containers (Postgres, MinIO, an old project), you get exactly this:

Container name	Docker image	Represents	Listens on
<code>ca-hospitala</code>	<code>hyperledger/fabric-ca:latest</code>	Hospital A's certificate authority	7054
<code>ca-hospitalb</code>	<code>hyperledger/fabric-ca:latest</code>	Hospital B's certificate authority	8054
<code>ca-lab</code>	<code>hyperledger/fabric-ca:latest</code>	Lab's certificate authority	9054
<code>ca-orderer</code>	<code>hyperledger/fabric-ca:latest</code>	Orderer org's certificate authority	10054
<code>orderer1.orderer.healthcare.com</code>	<code>hyperledger/fabric-orderer:latest</code>	The single ordering node	7050 (transactions), 7053 (admin)
<code>peer0.hospitala.healthcare.com</code>	<code>hyperledger/fabric-peer:latest</code>	Hospital A's peer	7051
<code>peer0.hospitalb.healthcare.com</code>	<code>hyperledger/fabric-peer:latest</code>	Hospital B's peer	8051
<code>peer0.lab.healthcare.com</code>	<code>hyperledger/fabric-peer:latest</code>	Lab's peer	9051
<code>proofoflife-hospitala</code>	<code>proofoflife_ccaas_image:latest</code>	Hospital A's copy of our chaincode	9999 (internal only)
<code>proofoflife-hospitalb</code>	<code>proofoflife_ccaas_image:latest</code>	Hospital B's copy of our chaincode	9999 (internal only)
<code>proofoflife-lab</code>	<code>proofoflife_ccaas_image:latest</code>	Lab's copy of our chaincode	9999 (internal only)

11 containers. 4 different Docker images. Only **one** of those 4 images (`proofoflife_ccaas_image`) was built from our own code — the other 3 (`fabric-ca` , `fabric-orderer` , `fabric-peer`) were downloaded pre-built from Hyperledger.

1.1 They all sit on one private Docker network

All 11 containers are attached to a single Docker network named `healthcare-fabric-net` (created by the first line of `network/docker-compose-ca.yaml`). This is what lets, say, `peer0.hospitala.healthcare.com` reach `orderer1.orderer.healthcare.com` just by that hostname — Docker runs an internal DNS resolver on this network so container names resolve to container IPs automatically. Nothing here is exposed to the real internet; it's all container-to-container traffic on your machine.



2. What a Docker image actually contains, per role

This section answers "what is literally inside these images" using facts read directly off the images on this machine (`docker inspect` , `docker history`), not documentation.

2.1 `hyperledger/fabric-ca` HYPERLEDGER'S CODE

Fact	Value
Image size	~223 MB
What it contains	A single compiled Go binary, <code>fabric-ca-server</code> , plus a minimal Linux base (Alpine) to run it in
Startup command	<code>fabric-ca-server start -b admin:adminpw</code> (from our <code>docker-compose-ca.yaml</code>)
What the process does	Listens on an HTTPS port for two kinds of requests: register ("create an identity record with this name/secret") and enroll ("here's my secret, give me a signed certificate")
Where its logic lives	Hyperledger's <code>fabric-ca</code> GitHub repository — we never see this source, only run the binary

This is the **only** container type we don't have running anymore in a way you can inspect live traffic on easily — it did its entire job during Step 2 (`register-enroll.sh`) and has been idle since. It's still running (so you could enroll more identities later, e.g. a doctor's identity in a future phase), but nothing is currently talking to it.

2.2 `hyperledger/fabric-orderer` HYPERLEDGER'S CODE

Fact	Value
Image size	~111 MB
What it contains	A single compiled Go binary, <code>orderer</code>
Startup command	<code>orderer start</code> (no arguments — all its configuration comes from ~25 environment variables we set in <code>docker-compose-net.yaml</code> , e.g. <code>ORDERER_GENERAL_LISTENPORT=7050</code>)
What the process does	Runs a gRPC server on port 7050 that accepts already-endorsed transactions, sequences them, and produces blocks. Runs a second gRPC server on port 7053 (the "channel participation admin API") for administrative commands like "start serving this channel," which is what <code>osnadmin</code> talks to
Where its logic lives	Hyperledger's <code>fabric</code> GitHub repository, package <code>orderer/</code> — specifically the Raft consensus implementation and the gRPC service definitions

2.3 `hyperledger/fabric-peer` **HYPERLEDGER'S CODE** — the most important one, covered in full in Section 4

Fact	Value
Image size	~142 MB
What it contains	A single compiled Go binary, <code>peer</code> , 59,731,256 bytes (~57 MB) by itself, at <code>/usr/local/bin/peer</code> inside the container — verified directly with <code>docker exec ... ls -la /usr/local/bin/</code>
Startup command	<code>peer node start</code>

2.4 `proofoflife_ccaas_image` **OUR CODE**

Fact	Value
Image size	28.4 MB — dramatically smaller than the others, because it's just our ~45-line Go program plus a minimal Alpine base, no Fabric internals at all
Built from	<code>chaincode/proofoflife/Dockerfile</code> , which we wrote (Section 5.5)
Startup command	<code>/proofoflife</code> — directly runs the compiled binary, no wrapper
What the process does	Opens a gRPC server on port 9999 and waits for a peer to connect and ask it to execute <code>Put</code> or <code>Get</code>
Where its logic lives	<code>chaincode/proofoflife/proofoflife.go</code> , in this repository — this is genuinely the only container out of all 11 whose entire application logic we personally wrote

3. Our code vs. Hyperledger's code — a hard line

This is worth stating as plainly as possible, because it's easy to lose track of after seeing dozens of files fly by.

3.1 Every file WE wrote, and what each one causes to happen

File (this repo)	What it is	What happens when it's used
<code>network/docker-compose-ca.yaml</code>	Docker Compose config	Starts 4 <code>fabric-ca-server</code> containers (Hyperledger's binary) with our chosen ports/names
<code>network/scripts/register-enroll.sh</code>	Bash script	Talks to those 4 CAs via the <code>fabric-ca-client</code> tool (also Hyperledger's, not ours) to create real certificates/keys on disk
<code>network/configtx/configtx.yaml</code>	Fabric config file	Consumed by <code>configtxgen</code> (Hyperledger's tool) to produce the channel's genesis block
<code>network/docker-compose-net.yaml</code>	Docker Compose config	Starts the 1 orderer + 3 peer containers (Hyperledger's binaries) with our chosen identities/ports
<code>network/scripts/join-channel.sh</code>	Bash script	Talks to the orderer's admin API and each peer's CLI to make them join the channel
<code>chaincode/proofoflife/proofoflife.go</code>	Actual Go application code	This is the only place in the whole system where we wrote real program logic — two functions, <code>Put</code> and <code>Get</code>
<code>chaincode/proofoflife/Dockerfile</code>	Docker build recipe	Compiles <code>proofoflife.go</code> into a binary and packages it into the 28.4MB image
<code>network/scripts/deploy-chaincode.sh</code>	Bash script	Packages/installs/approves/commits our chaincode using Hyperledger's <code>peer lifecycle chaincode</code> tool, then starts our 3 chaincode containers
<code>network/scripts/smoke-test.sh</code>	Bash script	Runs one real write + one real read using Hyperledger's <code>peer chaincode</code> tool

3.2 The honest tally

Out of everything running, we personally wrote: 4 Docker Compose/config files, 4 Bash orchestration scripts, and about **45 lines of Go** (the chaincode). Every byte of the actual peer, orderer, and CA logic — how a transaction gets endorsed, how Raft consensus works, how a block gets hashed and chained, how gRPC connections are authenticated — is Hyperledger's compiled software, which we configure and run but never edit. This is completely normal and is *the point* of using an established platform: you don't re-implement a consensus algorithm from scratch, you configure a battle-tested one correctly.

4. Deep dive: what IS a "peer," specifically

Since "what is the peer doing" was the core question, here it is in maximum concrete detail, using `peer0.hospitala.healthcare.com` as the example.

4.1 It is one process

Inside the container, there is exactly one running program: `/usr/local/bin/peer`, started with the arguments `node start`. That's it — not a suite of services, one binary, one process (though internally, like any real server, it runs many concurrent goroutines — Go's lightweight threads — to handle multiple connections at once).

4.2 How it gets configured (no code, just environment variables)

Every peer-specific fact it needs — its own identity, its address, which org it belongs to — is handed to it purely through environment variables set in `network/docker-compose-net.yaml`. For Hospital A's peer, the relevant ones are:

```
CORE_PEER_ID=peer0.hospitala.healthcare.com
CORE_PEER_ADDRESS=peer0.hospitala.healthcare.com:7051
CORE_PEER_LISTENADDRESS=0.0.0.0:7051
CORE_PEER_LOCALMSPID=HospitalAMSP
CORE_PEER_MSPCONFIGPATH=/etc/hyperledger/fabric/msp
CORE_PEER_TLS_CERT_FILE=/etc/hyperledger/fabric/tls/server.crt
CORE_PEER_TLS_KEY_FILE=/etc/hyperledger/fabric/tls/server.key
```

`CORE_PEER_MSPCONFIGPATH` is the important one for identity: it points inside the container at `/etc/hyperledger/fabric/msp`, which — via a volume mount also defined in `docker-compose-net.yaml` — is really the real folder on your host disk, `network/organizations/peer0organizations/hospitala.healthcare.com/peers/peer0.hospitala.healthcare.com/msp/`, produced back in Step 2 by `register-enroll.sh`. That folder contains the peer's actual private key and certificate. **This is how a generic prebuilt binary becomes "Hospital A's peer" specifically** — not by being compiled differently, but by being handed a specific identity folder at startup.

4.3 What it's listening for, on port 7051

Port 7051 is a **gRPC** server. gRPC is Google's remote procedure call framework — think of it as "call a function on another machine as if it were local," over HTTP/2, with strongly-typed messages defined in `.proto` files (Hyperledger defines these for every peer/orderer operation in the `fabric-protos` repository). Concretely, the peer exposes (among others) an **Endorser service** with essentially one method: `ProcessProposal(SignedProposal)` returns `(ProposalResponse)`.

When our `smoke-test.sh` ran `peer chaincode invoke --peerAddresses localhost:7051 ...`, what actually happened at the network level:

1. The `peer` CLI (running on your host, a completely separate process from the one inside the container — same binary, different invocation) opened a TLS-encrypted gRPC connection to `localhost:7051`
2. It sent one `ProcessProposal` call, whose payload was our signed proposal (chaincode name, function `Put`, args `["greeting", "hello from Hospital A"]`)
3. The peer process inside the container received this on its gRPC listener, verified the signature against the sending identity's cert, and then — separately — opened its **own** gRPC connection outward to `proofoflife-hospitala:9999` to actually run the chaincode logic
4. The peer got the result back from the chaincode container, signed it (this signature is the endorsement), and returned a `ProposalResponse` back over the original connection to the CLI

So a single peer container, during one `invoke`, is simultaneously a gRPC *server* (to the CLI on 7051) and a gRPC *client* (to the chaincode container on 9999). Both roles, same process.

4.4 What it does after endorsing: nothing, until a block arrives

After sending its endorsement back, the peer's involvement in that specific transaction pauses. It does not talk to the orderer directly for this step — the **client** (the `peer` CLI on your host) is the one that collects endorsements from multiple peers and forwards the whole bundle to the orderer. The peer's next involvement is passive: it also runs a **Deliver service**, through which the orderer pushes newly-created blocks out to every peer on the channel. When block N arrives via that connection, the peer:

1. Verifies the block's hash chain (Section 6.2 of our earlier conversation — recomputes the hash, checks it against the header)
2. Re-validates every transaction inside it (endorsement policy actually satisfied? any read/write conflict with something committed since?)
3. Applies valid transactions to its local copy of **world state** — which, concretely, is a key-value store on disk inside the container, at `/var/hyperledger/production` (mounted to a Docker volume named `peer0.hospitala.healthcare.com` so it survives container restarts)

4.5 Where the peer's send/receive logic actually lives (source code)

Since this was asked directly: it is **not anywhere in this repository**. It lives in Hyperledger Fabric's own Go source tree, roughly:

- `internal/peer/node/start.go` — what `peer node start` actually does on startup
- `core/endorser/endorser.go` — the `ProcessProposal` logic described above
- `core/chaincode/` — the code that manages connections out to chaincode containers (classic Docker-managed mode) or, in our CCaaS case, connects to an externally-running chaincode server
- `core/commmitter/` and `core/ledger/` — block validation and world-state application

We never open, edit, or even need to read these files to use Fabric correctly — but knowing they exist, and that they're what's actually executing inside that 59MB `peer` binary, should make "what is the peer doing" feel like a concrete, answerable question rather than a black box.

5. Every file we wrote, in the order it runs

5.1 `network/docker-compose-ca.yaml` [OUR CODE](#)

A Docker Compose file — YAML describing containers to run. Defines the shared network (`healthcare-fabric-net`) and 4 services, one per org, each running `hyperledger/fabric-ca:latest` with a different port and a different bind-mounted folder for its own data. Running `docker compose -f docker-compose-ca.yaml up -d` reads this file and asks the Docker daemon to create exactly what it describes — nothing here executes logic itself, it's declarative configuration for Docker.

5.2 `network/scripts/register-enroll.sh` [OUR CODE](#)

~130 lines of Bash. For each of the 4 orgs, it runs the `fabric-ca-client` command-line tool (Hyperledger's, pre-installed in `fabric-samples/bin`, not something we wrote) against that org's CA container, first as the bootstrap admin, then to register and enroll a peer/orderer identity and an org-admin identity. Every `fabric-ca-client enroll` call is a real HTTPS request to the CA container that comes back with a real certificate + private key, written to disk under `network/organizations/`.

5.3 `network/configtx/configtx.yaml` [OUR CODE](#)

Not a script — a declarative Fabric config file (covered in detail in our earlier conversation). Read by `configtxgen` (Hyperledger's tool) to produce `healthchannel.block`, the channel's genesis block.

5.4 `network/docker-compose-net.yaml` [OUR CODE](#)

Same idea as 5.1, but for the orderer + 3 peers. This is the file with the long `CORE_PEER_* / ORDERER_*` environment variable lists described in Section 4.2 — this is where a generic Fabric binary becomes "Hospital A's peer specifically," purely through configuration, not different code.

5.5 `chaincode/proofoflife/proofoflife.go` [OUR CODE — the only real application logic](#)

The entire file, for reference (this is genuinely all of it):

```
package main

import (
    "fmt"
    "os"

    "github.com/hyperledger/fabric-chaincode-go/v2/shim"
    "github.com/hyperledger/fabric-contract-api-go/v2/contractapi"
)

type SmartContract struct {
```

```

    contractapi.Contract
}

// Put writes a key/value pair to world state.
func (s *SmartContract) Put(ctx contractapi.TransactionContextInterface, key string, value
string) error {
    return ctx.GetStub().PutState(key, []byte(value))
}

// Get reads a key's current value from world state.
func (s *SmartContract) Get(ctx contractapi.TransactionContextInterface, key string)
(string, error) {
    value, err := ctx.GetStub().GetState(key)
    if err != nil {
        return "", fmt.Errorf("failed to read from world state: %w", err)
    }
    if value == nil {
        return "", fmt.Errorf("key %q does not exist", key)
    }
    return string(value), nil
}

func main() {
    chaincode, _ := contractapi.NewChaincode(&SmartContract{})
    server := &shim.ChaincodeServer{
        CCID:    os.Getenv("CHAINCODE_ID"),
        Address: os.Getenv("CHAINCODE_SERVER_ADDRESS"),
        CC:      chaincode,
        TLSProps: shim.TLSProperties{Disabled: true},
    }
    server.Start()
}

```

Note what we did *not* have to write: any networking code, any gRPC setup, any protocol handling.

`contractapi.NewChaincode` and `shim.ChaincodeServer` are Hyperledger's library code (imported, not copied) — they handle "listen on a port, speak the chaincode protocol, call my `Put / Get` methods when asked." Our contribution is purely the two business-logic methods; everything about *how* a peer reaches this code is handled by imported library code, one layer up from "written by us" but still compiled directly into our own 28.4MB binary (unlike the peer/orderer/CA, which are entirely separate prebuilt binaries we never compile ourselves).

5.6 `chaincode/proofoflife/Dockerfile` OUR CODE

```

FROM golang:1.25-alpine AS build
WORKDIR /src
COPY . .
RUN go build -o /proofoflife .

FROM alpine:latest
COPY --from=build /proofoflife /proofoflife
EXPOSE 9999
CMD ["/proofoflife"]

```

Two stages: the first has a full Go compiler and turns our `.go` file into a binary; the second is a nearly-empty Linux image that just holds that compiled binary and runs it. This is why the final image is only

28.4MB — it doesn't contain the Go compiler, source code, or any Fabric internals, just one small compiled program.

5.7 `network/scripts/join-channel.sh`, `deploy-chaincode.sh`, `smoke-test.sh` [OUR CODE](#)

All three are orchestration Bash scripts — they don't implement blockchain logic themselves, they call Hyperledger's own command-line tools (`osnadmin`, `peer channel`, `peer lifecycle chaincode`, `peer chaincode`) with the right identities and addresses set as environment variables first. Full behavior of each was walked through message-by-message in our previous conversation.

6. Tracing one real request end to end, with real names

This is the exact sequence from running `smoke-test.sh`, with every hop named concretely.

```
YOUR HOST TERMINAL
  `peer chaincode invoke ...` (the peer CLI binary, run directly, NOT in a container)
    |
    | gRPC/TLS to localhost:7051
    v
peer0.hospitala.healthcare.com (container, port 7051)
  | verifies signature against HospitalAMSP root cert
  | gRPC to proofoflife-hospitala:9999 (internal Docker network)
  v
proofoflife-hospitala (container, port 9999) -- OUR CODE runs here
  | executes Put("greeting", "hello from Hospital A")
  | returns proposed write, does NOT touch real world state yet
  ^
  | result flows back up
peer0.hospitala.healthcare.com
  | signs the result (= its endorsement)
  v
(same request also sent to peer0.hospitalb.healthcare.com:8051,
 which independently repeats the same steps against
 proofoflife-hospitalb:9999, and produces a second endorsement)

YOUR HOST TERMINAL
  now holds 2 signed endorsements
    |
    | gRPC/TLS to localhost:7050
    v
orderer1.orderer.healthcare.com (container, port 7050)
  | sequences the transaction, builds a block
  | pushes the new block to ALL peers via their Deliver service
  v
peer0.hospitala.healthcare.com  peer0.hospitalb.healthcare.com  peer0.lab.healthcare.com
  | each independently validates the block
  | each writes "greeting" into its own local world state (on-disk, per container)

--- 3 seconds later ---

YOUR HOST TERMINAL
  `peer chaincode query ...` targeting ONLY peer0.lab.healthcare.com:9051
    |
    | gRPC/TLS to localhost:9051 -- no orderer involved, purely local read
    v
peer0.lab.healthcare.com
  | looks up "greeting" in ITS OWN on-disk world state
  | returns "hello from Hospital A"
```

The Lab's peer was never asked to endorse anything and never connected to `proofoflife-lab` for this transaction at all — it only received the finished block from the orderer and applied it. That it can still answer the query correctly is the actual proof that replication works, not just that one peer remembers what it was told directly.

7. Quick-reference glossary (grounded in what you've actually seen)

Term	In this system, concretely
Peer	The <code>hyperledger/fabric-peer</code> container, e.g. <code>peer0.hospitala.healthcare.com</code> — a 59MB Go binary running as one process, listening on a gRPC port (7051/8051/9051)
Orderer	The <code>hyperledger/fabric-orderer</code> container, <code>orderer1.orderer.healthcare.com</code> — listens on 7050 for endorsed transactions, 7053 for admin commands
CA (Certificate Authority)	One of the 4 <code>hyperledger/fabric-ca</code> containers — issued every certificate/key currently sitting under <code>network/organizations/</code>
MSP (Membership Service Provider)	Not a running process — a folder structure (e.g. <code>network/organizations/peerOrganizations/hospitala.healthcare.com/msp/</code>) containing certs, that Fabric reads to know "who is allowed to claim to be HospitalAMSP"
Chaincode	Our <code>proofoflife</code> program — in this deployment mode, 3 separate running containers (one per org), each a copy of the same 28.4MB image
Channel	<code>healthchannel</code> — not a container, a shared identity all 4 orgs' processes agree to recognize, first defined by the genesis block
World state	A key-value store, physically on-disk inside each peer container (Docker volume), holding current values like <code>greeting = "hello from Hospital A"</code>
Ledger / blocks	The full ordered history, also on-disk per peer, immutable once written — world state is derived from replaying valid transactions from the ledger
Endorsement	A cryptographic signature from a peer, over a proposed transaction result — collected by the client (the <code>peer</code> CLI) from ≥ 2 of the 3 orgs' peers before submission to the orderer
gRPC	The actual network protocol every one of these containers use to talk to each other — HTTP/2-based remote procedure calls, defined by Hyperledger's <code>.proto</code> files

This document describes Phase 2 of the project as committed at `abd945b`. It will go stale as later phases add real chaincode, a Go backend, and application-level components — treat it as a snapshot of "how the network layer works," not a living reference.